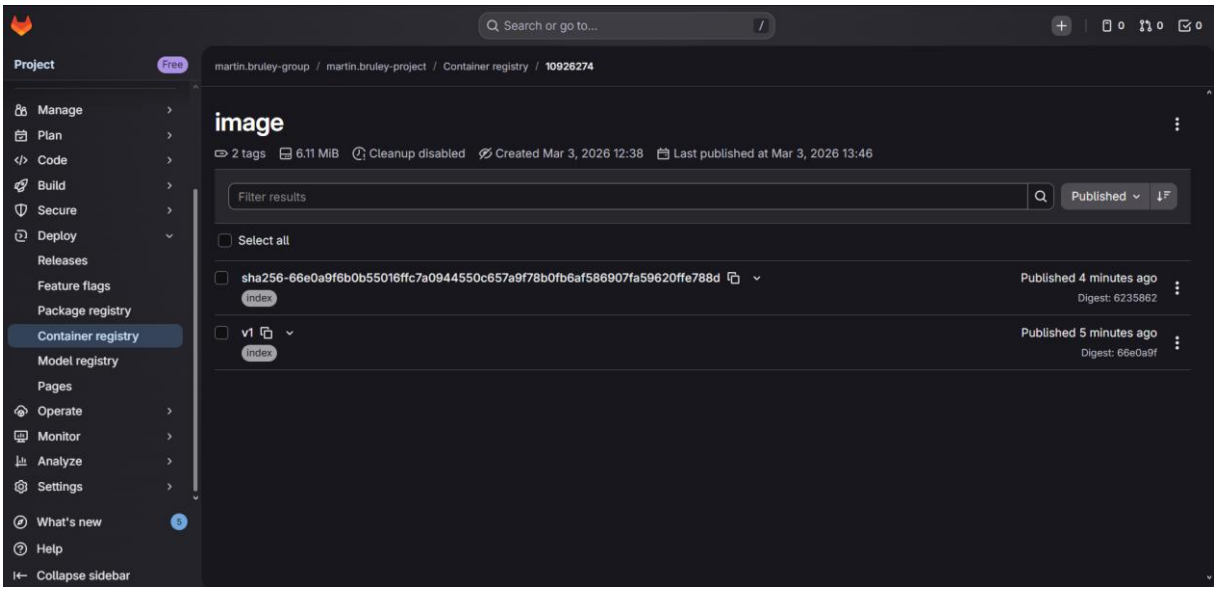
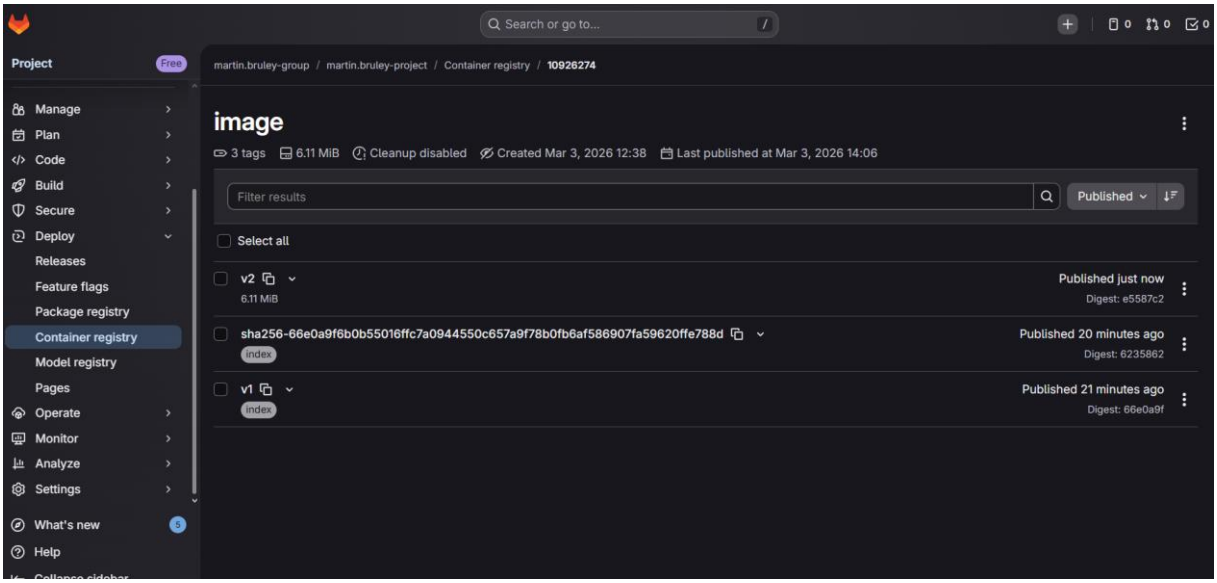
Here, we can observe that the signature was successful, with the “index” showing us the signature and v1 link.



After adding modified v1 (v2), we can see the signature is not assigned to it.



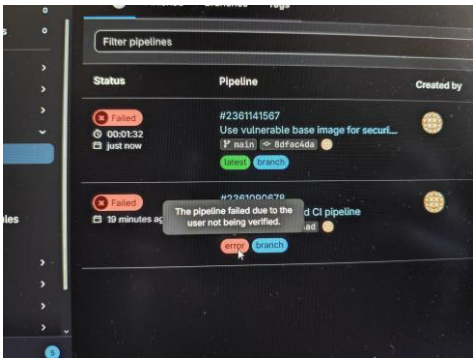
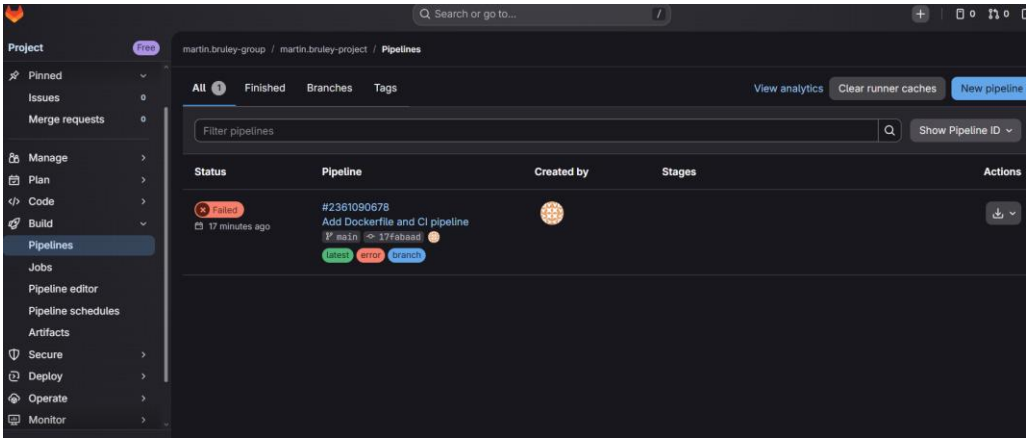
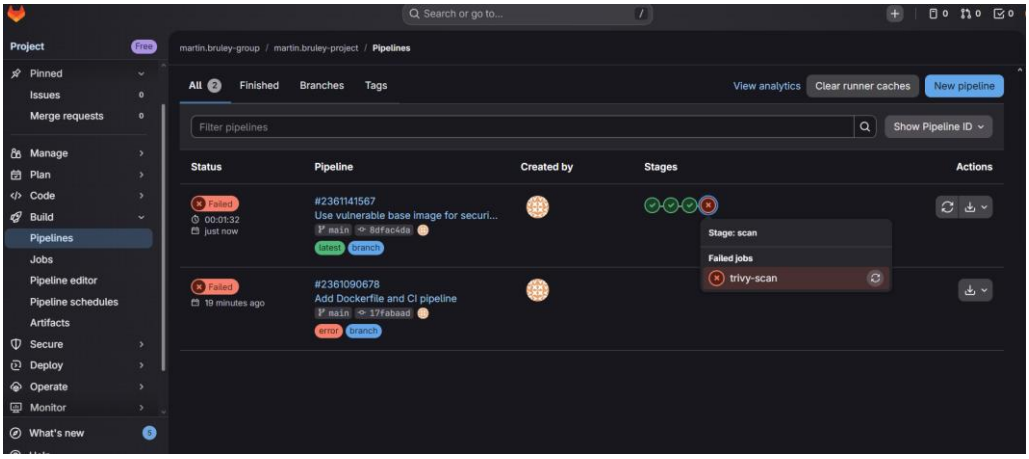
Indeed the cosign check only validates v1 authenticity

```
marti@HONORDeMartin:~/martin.bruley-project$ ./cosign verify --key cosign.pub registry.gitlab.com/martin.bruley-group/ma
rtin.bruley-project/image:v1

Verification for registry.gitlab.com/martin.bruley-group/martin.bruley-project/image:v1 --
The following checks were performed on each of these signatures:
  - The cosign claims were validated
  - Existence of the claims in the transparency log was verified offline
  - The signatures were verified against the specified public key

[{"critical":{"identity":{"docker-reference":"registry.gitlab.com/martin.bruley-group/martin.bruley-project/image:v1"},"
image":{"docker-manifest-digest":"sha256:66e0a9f6b0b55016ffc7a0944550c657a9f78b0fb6af586907fa59620ffe788d"},"type":"http
s://sigstore.dev/cosign/sign/v1"},"optional":{}}]
marti@HONORDeMartin:~/martin.bruley-project$ ./cosign verify --key cosign.pub registry.gitlab.com/martin.bruley-group/ma
rtin.bruley-project/image:v2
Error: no signatures found
error during command execution: no signatures found
marti@HONORDeMartin:~/martin.bruley-project$
```

The pipeline is used to impose an order of several verifications :

Lint checks the quality and security of the Dockerfile, then **build** builds the image and signs it with cosign. **Verify** checks that the cosign signature is valid and finally, **scan** uses trivy to scan the image for vulnerabilities (which explains why our pipeline fails at that step when pushing the vulnerable image :

“FROM alpine:3.12

RUN apk add --no-cache curl=7.79.1-r1 »

Note : the first Pipeline was not verified because I hadn’t verified my gitlab account, but it turned out to work just fine after that.